



# You Only Look Once: Unified, Real-Time Object Detection

|          |                                                                                 |
|----------|---------------------------------------------------------------------------------|
| 📅 기간     | @05/28/2025 → 06/03/2025                                                        |
| 🕒 주차     | 13주차                                                                            |
| 📎 논문     | <a href="https://arxiv.org/pdf/1506.02640">https://arxiv.org/pdf/1506.02640</a> |
| ☀️ 상태    | 완료                                                                              |
| ☑️ 예습/복습 | 예습과제                                                                            |
| ≡ 참고 자료  | <a href="#">참고 블로그1</a><br><a href="#">참고 블로그2</a>                              |

## 0. Abstract

### 1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

### 2. Unified Detection

2.1 Network Design

2.2 Training

2.3 Inference

2.4 Limitations of YOLO

### 3. Comparison to Other Detection Systems

### 4. 실험 및 결과

4.1 Comparison to Other Real-Time Systems

4.2 VOC 2007 Error Analysis

4.3 Combining Fast R-CNN and YOLO

4.4 VOC 2012 Results

4.5 Generalizability: Person Detection in Artwork

### 5. Real-Time Detection In The Wild

### 6. Conclusion

## 0. Abstract

- 객체 탐지를 위한 새로운 접근 방식 → **YOLO** 소개

- 기존의 객체 탐지 연구들 → **분류기(classifier)**를 객체 탐지에 **재활용**하는 방식
- YOLO → 객체 탐지를 **공간적으로 분리된 바운딩 박스**와 이에 연관된 **클래스 확률**로 **회귀 문제(regression problem)**로 재정의!
  - 하나의 신경망(neural network) 사용하여 전체 이미지를 단 한 번 평가하여 바운딩 박스와 클래스 확률 직접 예측함.
  - 탐지 파이프라인 전체가 단일 네트워크로 구성 → 탐지 성능을 기준으로 끝에서 끝까지(end-to-end) 최적화할 수 있음.
  - 속도 측면
    - 기본 YOLO 모델 : 초당 45프레임(fps)의 실시간 속도로 이미지 처리
    - 더 적은 버전 fast YOLO : 초당 155프레임(fps) 처리하면서도 다른 실시간 탐지기보다 2배 높은 mAP 기록함.
  - 최신 객체 탐지 시스템들보다 위치 정확도는 낮음.
    - 그러나 배경에서의 false positive 덜 발생함.
  - 객체에 대하여 일반화된 표현(General Representation) 효과적으로 학습
    - 자연 이미지에서 학습한 모델이 예술 작품 등의 도메인에서도 DPM이나 R-CNN보다 더 나은 성능 보여줌.

## 1. Introduction

### 논문이 다루는 분야

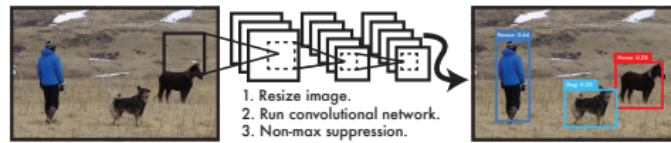
- 연구 분야 : 실시간 객체 탐지(Real-Time Object Detection)
  - 빠르고 정확한 객체 탐지 알고리즘이 존재한다면?
    - 컴퓨터가 별도의 센서 없이도 운전할 수 있음.
    - 시각 보조 장치가 실시간 장면 정보 사용자에게 전달 가능.
    - 일반 목적의 반응형 로봇 시스템에도 새로운 가능성 열림.

### 해당 task에서 기존 연구 한계점

- 기존 객체 탐지 시스템 → **분류기를 재활용**하여 탐지 수행
  - 객체 탐지 위해 특정 객체의 분류기를 이미지 내 여러 위치와 스케일에 걸쳐 반복적으로 실행.
    - ex) DPM(Deformable Parts Model) : 전체 이미지를 **일정 간격으로 슬라이딩 윈도우 방식**으로 훑으며 분류기 실행.
- 더 최근 접근 방식 → R-CNN
  - **region proposal 기법** 사용하여 잠재적 바운딩 박스 생성한 뒤, 각 제안된 박스에 분류기 실행함.
  - 분류 이후, 바운딩 박스 다듬고, 중복 제거, 다른 객체와의 관계에 따라 점수 조정하는 등의 **복잡한 후처리 과정**

⇒ 느림, 최적화 어려움, 각 구성요소를 별도로 훈련해야 한다는 단점

## 논문의 contributions



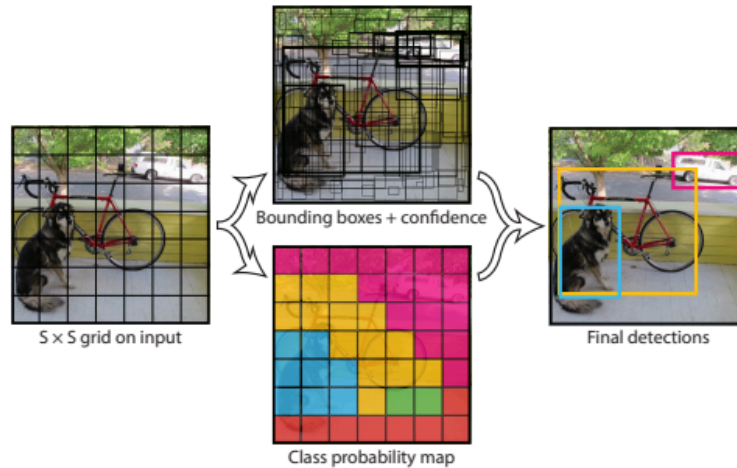
**Figure 1: The YOLO Detection System.** Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to  $448 \times 448$ , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

매우 단순함. 하나의 합성곱 신경망이 여러 바운딩 박스와 그에 대한 클래스 확률 동시에 예측함.

- 본 논문은 객체 탐지를 **단일 회귀 문제로 재정의**
    - 이미지 픽셀 정보로부터 바운딩 박스 좌표와 클래스 확률을 직접 예측함.
    - 이미지에 대해 **단 한 번만 살펴보며(You Only Look Once)**, 그 안에 어떤 객체가 있는지를 예측함!
  - **장점(이점)**
    1. **매우 빠름.**
      - 탐지를 회귀 문제로 정의 → 복잡한 파이프라인 필요 없음.
      - 이미지 하나를 테스트할 때 단순히 신경망에 1번 넣기만 하면 됨.
        - Titan X GPU에서 기본 네트워크 45fps, 빠른 버전은 150fps 동작.
        - 스트리밍 영상도 25ms 미만의 지연 시간으로 실시간 처리 가능함.
    2. 예측할 때 이미지 전체를 **전역적(global reasoning)**으로 고려함.
      - 슬라이딩 윈도우 & region proposal 기반 기법과 달리, 학습 및 테스트 **전체 이미지 활용**하여 **클래스와 외형에 대한 문맥 정보 암묵적으로 내포**할 수 있음.
        - Fast R-CNN : 이미지의 배경 패치를 객체로 오인.
        - **YOLO** : 배경 오탐(background errors) 절반 이하로 줄어듦.
    3. 객체에 대한 **범용적 표현 능력** 갖추.
      - 자연 이미지로 학습하고 예술 작품 같은 도메인에서 테스트할 경우, DPM/R-CNN보다 훨씬 나은 성능 보여줌.
      - 예상치 못한 입력이나 새로운 도메인에서도 잘 작동함.
- 👉 정확도 면에서 최첨단 시스템보다 뒤처지는 부분
- 특히, 작은 객체 정확히 찾아내는 것 어려움.

## 2. Unified Detection

- 객체 탐지의 개별 구성 요소  $\Rightarrow$  **하나의 신경망으로 통합!**
  - 전체 이미지로부터 얻은 특징(feature) 사용하여 각각 바운딩 박스 예측함.
  - 이미지 내 모든 이미지 클래스에 대해 모든 바운딩 박스를 동시에 예측함.
- $\Rightarrow$  **전체 이미지 + 그 안의 모든 객체들에 대한 전역적인 추론** 수행함.
- end-to-end 훈련과 실시간 처리 속도를 가능하게 하면서도 높은 average precision 유지함.



**Figure 2: The Model.** Our system models detection as a regression problem. It divides the image into an  $S \times S$  grid and for each grid cell predicts  $B$  bounding boxes, confidence for those boxes, and  $C$  class probabilities. These predictions are encoded as an  $S \times S \times (B * 5 + C)$  tensor.

### [Detection 방식]

1. Input 이미지를  $S \times S$  크기의 Grid로 나눔.
  - 한 Grid cell 안에 **어떤 객체의 중심이 포함되면**, 해당 Grid cell이 그 객체에 대해 책임 지고 탐지 수행함.
2. 각 Grid cell은  $B$ 개의 바운딩 박스와 Confidence 스코어를 예측함.

$$Pr(Object) * IOU_{pred}^{truth}$$

- **Confidence Score** : 박스 안에 객체가 포함될 확률, 박스가 객체를 얼마나 잘 담고 있는지에 대한 곱으로 표현.
  - $Pr(Object) \rightarrow$  객체가 포함되면 1, 없으면 0.
    - 객체가 포함되지 않은 경우 전체가 0이 됨.
    - 점수 높기 위해서는 예측한 바운딩 박스가 최대한 Ground Truth box와 겹치도록 하여 1에 가까워야 함.

3. 각 Grid cell은 C개의 클래스에 대한 Conditional Class Probabilities 계산.

- 각 바운딩 박스 안에 있는 객체가 어떤 클래스일지에 대한 조건부 확률.

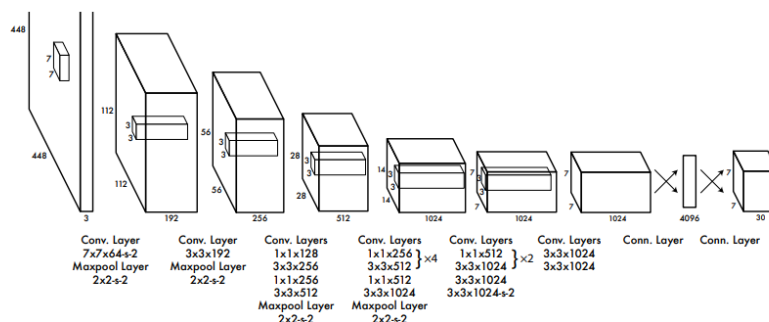
$$Pr(Class_i|Object)$$

- 최종적으로 구하는 값 → **Class-specific Confidence Score**
  - 바운딩 박스 안에 특정 객체가 속할 확률 & 바운딩 박스가 객체를 얼마나 잘 담고 있는지에 대한 정보 표현하는 값.

$$Pr(Class_i|Object) * Pr(Object) * IOU_{pred}^{truth} = Pr(Class_i) * IOU_{pred}^{truth}$$

- 회귀 문제로 모델링
  - 이미지를  $S \times S$  그리드로 나눈 후, 각 셀이 B개의 바운딩 박스와 C개의 클래스 확률을 예측. → 하나의  $S \times S \times (B \times 5 + C)$  크기의 텐서로 표현.
- PASCAL VOC 기준 설정:
  - $S = 7, B = 2, C = 20$
  - 결과적으로 최종 출력은  $7 \times 7 \times 30$  텐서

## 2.1 Network Design



**Figure 3: The Architecture.** Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating  $1 \times 1$  convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution ( $224 \times 224$  input image) and then double the resolution for detection.

- CNN 구현, PASCAL VOC 데이터셋에서 성능 평가.
  - 앞단의 CNN 레이어 → 이미지로부터 특징 추출
  - 뒷단의 fully connected layers → 클래스 확률과 바운딩 박스 좌표 예측
- GoogLeNet 기반으로 함.
  - 총 **24개의 합성곱 층(convolution layers)**
  - 이어지는 **2개의 완전 연결 층**으로 구성.
    - GoogLeNet의 Inception 모듈 대신, **1×1 축소 층과 3×3 합성곱 층**을 조합
- Fast YOLO

- 9개의 합성곱 층, 더 적은 필터 사용하여 속도 극대화

## 2.2 Training

### <사전 훈련(pretraining)>

- ImageNet 1000 클래스 분류 데이터셋에서 224×224 이미지로 수행
- 상위 20개 합성곱 층 + 평균 풀링 + FC층을 약 1주일간 학습
- ImageNet 검증셋에서 top-5 정확도 88%
- 프레임워크: Darknet

### <탐지용 전환(detection adaptation)>

- 새로운 4개의 합성곱 층 + 2개의 FC층 추가
- 입력 해상도는 224×224에서 **448×448**로 증가
- 최종 레이어는 바운딩 박스 좌표 및 클래스 확률을 예측

### <정규화 방식>

- $x, y, w, h$ : 이미지 크기에 대해 정규화 (0~1 범위)
- 활성화 함수
  - 최종 레이어는 선형(linear)
  - 나머지는 Leaky ReLU

### <손실 함수 및 학습 설정>

- **손실 함수: 제곱 오차 합(sum-squared error)**
  - 탐지 성능 최적화와는 다소 차이 있으나 구현 용이
- 문제점과 해결
  - 객체가 없는 셀의 confidence 오차가 지나치게 크면 학습 불안정  
→ 가중치 조정
    - $\lambda_{\text{coord}} = 5$  (좌표 오차 강조)
    - $\lambda_{\text{noobj}} = 0.5$  (배경 confidence 오차 축소)
  - 작은 박스일수록 위치 오차에 민감  
→  $w, h$  대신  $\sqrt{w}, \sqrt{h}$ 를 예측하여 균형 맞춤
- **책임 할당**
  - 하나의 객체에 대해 가장 IOU가 높은 바운딩 박스 예측기에 책임 할당
  - 예측기가 **특정 객체 유형/크기에 특화**되도록 유도
- **학습 하이퍼파라미터**
  - epoch: 약 135회

- 배치 크기: 64
- momentum: 0.9
- weight decay: 0.0005
- learning rate 스케줄:
  - 처음에는 천천히  $10^{-3} \rightarrow 10^{-2}$
  - 이후 75 epoch :  $10^{-2}$
  - 다음 30 epoch :  $10^{-3}$
  - 마지막 30 epoch :  $10^{-4}$
- 정규화 기법
  - dropout(0.5) 사용
  - data augmentation:
    - 최대 20% 랜덤 크기 변경 및 이동
    - HSV 색공간에서 명도와 채도를 1.5배까지 랜덤 조정

## 2.3 Inference

- 테스트 시에도 학습과 동일하게 한 번의 네트워크 실행만 필요
  - PASCAL VOC 기준, YOLO는 **98개의 바운딩 박스**와 각 박스에 대한 클래스 확률을 예측.
  - YOLO는 단일 네트워크 실행만으로 매우 빠르게 동작.
- grid 구조 덕분에 **공간적 다양성 보장**, 대부분의 경우 **한 객체에 대해 하나의 셀만 예측**
  - 객체가 너무 크거나 경계에 걸쳐 있으면 여러 셀이 예측할 수 있음.
  - 이를 해결하기 위해 Non-Maximum Suppression(NMS) 적용하면 mAP 2~3% 정도 증가함.

## 2.4 Limitations of YOLO

1. 각 grid cell이 **두 개의 박스만 예측하고, 하나의 클래스만 포함**할 수 있음.
  - 가까이 붙어 있는 다수의 작은 객체 탐지에 제한
2. 다양한 비율이나 구성의 객체에는 일반화 성능이 낮음.
3. 네트워크 구조상, 입력 이미지를 반복적으로 downsampling함.
  - 바운딩 박스 예측에 사용되는 특징이 다소 거침.(coarse features) ⇒ 입력 이미지에 비해 낮은 해상도를 가진 특징 맵.(정밀도 부족)
4. 손실 함수는 작은 박스와 큰 박스의 오차를 동등하게 취급함.
  - 작은 박스일수록 오차의 영향이 큼.
  - 전체 오류의 주된 원인은 위치 오류(Localization error)

### 3. Comparison to Other Detection Systems

- 탐지 파이프라인 → 일반적으로 입력 이미지로부터 강건한 특징(robust features) 추출하는 것으로 시작
- 그 다음, 분류기(classifier) 또는 국소화기(localizer) 사용하여 특징 공간에서 객체 식별함.  
→ 전체 이미지에 대해 **슬라이딩 윈도우** 방식으로 적용되거나, 이미지 내 **일부 영역에만** 적용됨.

#### ▼ Deformable Parts Models (DPM)

- 슬라이딩 윈도우 기반의 객체 탐지 방식 사용함.
- 고정된 파이프라인으로 구성, 정적인 특징 추출, 영역 분류, 높은 점수를 받은 영역에 대한 바운딩 박스 예측 등을 순차적으로 수행함.

⇒ YOLO : 이러한 분리된 처리 과정을 **하나의 합성곱 신경망**으로 대체!

- 특징 추출, 바운딩 박스 예측, NMS, 문맥 정보 추론 등을 동시에 수행.
- 고정된 특징 대신 탐지 작업에 맞춰 훈련된 특징(feature) 직접 학습함.
  - DPM보다 더 빠르고, 더 정확한 모델 만들 수 있음.

#### ▼ R\_CNN

- 슬라이딩 윈도우 대신 region proposal 사용하여 이미지 내 객체 후보 영역 찾음.
- Selective Search로 후보 바운딩 박스 생성하고, CNN이 특징 추출, SVM이 각 박스 평가, 선형 모델(linear model)이 바운딩 박스 조정, NMS로 중복 예측 제거함.
  - 각 단계는 별도로 정교하게 조정해야 하며, 전체 시스템은 매우 느림.

⇒ YOLO : 각 Grid Cell이 바운딩 박스 제안, CNN 특징 사용하여 그 박스 평가.(유사점)

- 그러나, YOLO는 **spatial constraints** 걸어, **같은 객체를 여러 번 중복 탐지하는 문제 줄임**.

#### ▼ 기타 실시간 탐지기 (Other Fast Detectors)

- Fast R-CNN, Faster R-CNN : R-CNN 구조의 속도 개선하려는 시도
  - 합성곱 연산 공유하거나, region proposal을 Selective Search 대신 신경망으로 대체함.
    - R-CNN에 비해 속도와 정확도 모두 향상, 여전히 실시간(real-time) 성능 충족하지 못함.
  - DPM 파이프라인의 속도를 높이려는 다양한 시도들
    - HOG 연산 속도 개선, Cascade 도입, GPU로 연산을 오프로드함.
    - 실제로 실시간 수준인 경우는 **30Hz DPM**이 유일.

⇒ YOLO : 기존의 탐지 파이프라인을 **부분 최적화**하는 대신, 아예 **탐지 전체를 처음부터 끝까지 단순화, 빠르게 설계함**.

#### ▼ 단일 클래스 탐지기 (Single-Class Detectors)

- 얼굴이나 사람처럼 단일 클래스만 탐지하는 모델은 변동성 적음. → 최적화 쉬움.



⇒ YOLO : 다양한 클래스 동시에 탐지해야 하는 **범용 탐지기(general-purpose detector)**, 다양한 객체들에 대한 탐지 능력 갖추고 있음.

#### ▼ Deep MultiBox

- Selective Search 대신 CNN을 사용해 객체 후보 영역을 예측하는 **MultiBox**를 제안.
  - 객체 존재 확률 대신 단일 클래스만 예측하도록 수정하여 단일 객체 탐지 가능하지만, 일반 객체 탐지 불가능함.
  - 탐지 파이프라인 일부 기능만 담당, 추가적인 이미지 패치 분류가 필요함.

⇒ YOLO : MultiBox → CNN 이용하여 바운딩 박스 예측한다는 점에서 유사, **단일 모델만으로 탐지 완료하는 완전한 시스템!**

#### ▼ OverFeat

- CNN 이용하여 물체의 위치 찾는 localization 수행하고, 이를 기반으로 탐지 수행하는 OverFeat 제안.
  - 슬라이딩 윈도우 기반 탐지를 효율적으로 수행, 여전히 분리된(local) 방식
  - 탐지 성능이 아닌 위치 정확도만을 최적화함.
  - 예측 시 국소 정보(local information)만 보고 판단하므로, 전역 문맥 이해하지 못함.
    - 후처리 과정 매우 중요함.

#### ▼ MultiGrasp

- YOLO의 구조 → **MultiGrasp** 시스템에서 착안.
  - Grid 기반 바운딩 박스 회귀 → **MultiGrasp 구조 차용**
  - 그러나, grasp detection은 객체 탐지보다 훨씬 간단한 구조
    - MultiGrasp : 이미지 하나에 하나의 graspable 영역만 예측하면 됨.
      - 객체 크기, 위치, 경계, 클래스 고려하지 x

⇒ YOLO : 여러 객체에 대한 바운딩 박스와 클래스 확률 동시에 예측.

## 4. 실험 및 결과

- **PASCAL VOC 2007 데이터셋**
  - YOLO & 다른 실시간 탐지 시스템 비교

## ? 🤔 VOC 데이터셋이란?

- PASCAL VOC (Visual Object Classes)
  - 출처 : 영국의 PASCAL(성능 평가 챌린지) 컨소시엄에서 주관
  - 기간 : 2005년부터 2012년까지 매년 공개
  - 목적 : 다양한 객체 인식 알고리즘의 성능을 공정하게 비교할 수 있는 표준 데이터셋 제공
  - 포함된 task

| Task                  | 설명                           |
|-----------------------|------------------------------|
| Classification        | 이미지에 특정 클래스가 존재하는지 분류        |
| Detection             | 객체의 위치(Bounding Box)와 클래스 예측 |
| Segmentation          | 객체의 픽셀 단위 경계 예측              |
| Action Classification | 사람의 행동을 인식                   |

## 4.1 Comparison to Other Real-Time Systems

- 많은 객체 탐지 연구 → 전통적인 탐지 파이프라인 속도를 개선하는 데 집중
  - 실제로 30fps 이상 달성한 모델은 GPU 기반 DPM뿐.

| Real-Time Detectors     | Train     | mAP         | FPS        |
|-------------------------|-----------|-------------|------------|
| 100Hz DPM [31]          | 2007      | 16.0        | 100        |
| 30Hz DPM [31]           | 2007      | 26.1        | 30         |
| Fast YOLO               | 2007+2012 | 52.7        | <b>155</b> |
| YOLO                    | 2007+2012 | <b>63.4</b> | 45         |
| Less Than Real-Time     |           |             |            |
| Fastest DPM [38]        | 2007      | 30.4        | 15         |
| R-CNN Minus R [20]      | 2007      | 53.5        | 6          |
| Fast R-CNN [14]         | 2007+2012 | 70.0        | 0.5        |
| Faster R-CNN VGG-16[28] | 2007+2012 | 73.2        | 7          |
| Faster R-CNN ZF [28]    | 2007+2012 | 62.1        | 18         |
| YOLO VGG-16             | 2007+2012 | 66.4        | 21         |

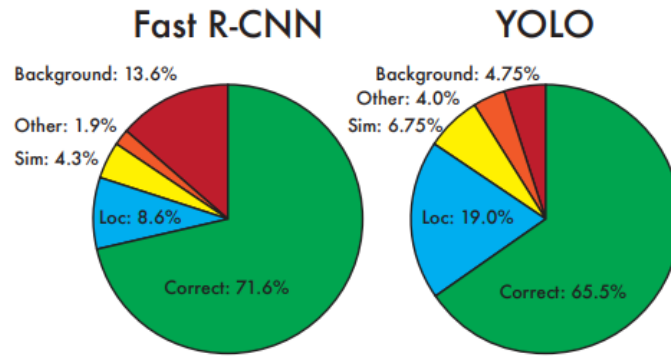
**Table 1: Real-Time Systems on PASCAL VOC 2007.** Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

- YOLO : PASCAL VOC 기준, 가장 빠른 탐지기.
  - Fast YOLO : 기존 실시간 탐지기보다 2배 이상 정확함.
- VGG-16 기반으로 훈련한 결과
  - 정확도는 더 높지만 속도가 느림.

- 다른 VGG-16 기반 탐지기와의 비교를 위해 사용, 본 논문에서는 빠른 모델들에 초점 맞춤.

## 4.2 VOC 2007 Error Analysis

- YOLO & Fast R-CNN → 서로 다른 유형의 오류를 보임.



**Figure 4: Error Analysis: Fast R-CNN vs. YOLO** These charts show the percentage of localization and background errors in the top N detections for various categories (N = # objects in that category).

- **Correct** : 올바른 클래스 + IOU > 0.5
- **Localization** : 클래스는 맞음, IOU는 0.1 ~ 0.5
- **Similar** : 유사한 클래스, IOU > 0.1
- **Other** : 다른 클래스, IOU > 0.1
- **Background** : IOU < 0.1 (아무 객체도 없음)

<결과>

- YOLO : **localization** 오류가 많음. (위치가 부정확)
- Fast R-CNN : **배경에 잘못된 탐지** 더 자주 함. (false positive)

## 4.3 Combining Fast R-CNN and YOLO

- YOLO → 배경에 대한 오탐이 적기 때문에, **Fast R-CNN의 배경 오류 걸러내는 필터 역할** 활용 가능
- 방법
  - **Fast R-CNN이 예측한 바운딩 박스 중 YOLO도 유사한 위치 탐지했다면, 그 박스의 점수를 YOLO 확률 및 IOU 기반으로 보정**

|                        | mAP         | Combined    | Gain       |
|------------------------|-------------|-------------|------------|
| Fast R-CNN             | 71.8        | -           | -          |
| Fast R-CNN (2007 data) | <b>66.9</b> | 72.4        | .6         |
| Fast R-CNN (VGG-M)     | 59.2        | 72.4        | .6         |
| Fast R-CNN (CaffeNet)  | 57.1        | 72.1        | .3         |
| YOLO                   | 63.4        | <b>75.0</b> | <b>3.2</b> |

**Table 2: Model combination experiments on VOC 2007.** We examine the effect of combining various models with the best version of Fast R-CNN. Other versions of Fast R-CNN provide only a small benefit while YOLO provides a significant performance boost.

⇒ YOLO 활용하면 Fast R-CNN 성능 크게 향상시킬 수 있음!

- 단, 두 모델을 따로 실행한 후 결합하므로 **YOLO의 속도 이점은 직접적으로 활용 x**
- 하지만 YOLO는 빠르기 때문에 **총 계산 시간에 큰 영향을 주지 않음.** (연산 시간 늘어나진 않음.)

## 4.4 VOC 2012 Results

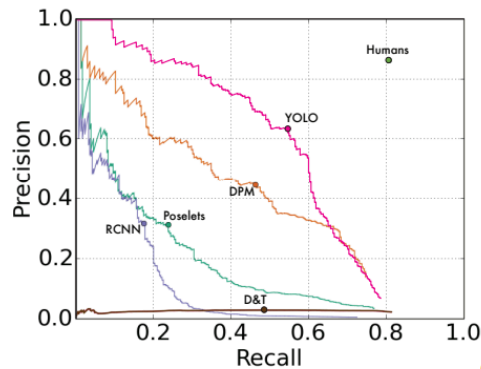
| VOC 2012 test            | mAP         | aero        | bike        | bird        | boat        | bottle      | bus         | car         | cat         | chair       | cow         | table       | dog         | horse       | mbike       | person      | plant       | sheep       | sofa        | train       | tv          |
|--------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| MR_CNN_MORE_DATA [11]    | <b>73.9</b> | <b>85.5</b> | <b>82.9</b> | <b>76.6</b> | <b>57.8</b> | <b>62.7</b> | <b>79.4</b> | 77.2        | 86.6        | <b>55.0</b> | <b>79.1</b> | <b>62.2</b> | 87.0        | <b>83.4</b> | <b>84.7</b> | 78.9        | 45.3        | 73.4        | 65.8        | 80.3        | 74.0        |
| HyperNet_VGG             | 71.4        | 84.2        | 78.5        | 73.6        | 55.6        | 53.7        | 78.7        | <b>79.8</b> | 87.7        | 49.6        | 74.9        | 52.1        | 86.0        | 81.7        | 83.3        | <b>81.8</b> | <b>48.6</b> | <b>73.5</b> | 59.4        | 79.9        | 65.7        |
| HyperNet_SP              | 71.3        | 84.1        | 78.3        | 73.3        | 55.5        | 53.6        | 78.6        | 79.6        | 87.5        | 49.5        | 74.9        | 52.1        | 85.6        | 81.6        | 83.2        | 81.6        | 48.4        | 73.2        | 59.3        | 79.7        | 65.6        |
| <b>Fast R-CNN + YOLO</b> | <b>70.7</b> | <b>83.4</b> | <b>78.5</b> | <b>73.5</b> | <b>55.8</b> | <b>43.4</b> | <b>79.1</b> | <b>73.1</b> | <b>89.4</b> | <b>49.4</b> | <b>75.5</b> | <b>57.0</b> | <b>87.5</b> | <b>80.9</b> | <b>81.0</b> | <b>74.7</b> | <b>41.8</b> | <b>71.5</b> | <b>68.5</b> | <b>82.1</b> | <b>67.2</b> |
| MR_CNN_S_CNN [11]        | 70.7        | 85.0        | 79.6        | 71.5        | 55.3        | 57.7        | 76.0        | 73.9        | 84.6        | 50.5        | 74.3        | 61.7        | 85.5        | 79.9        | 81.7        | 76.4        | 41.0        | 69.0        | 61.2        | 77.7        | 72.1        |
| Faster R-CNN [28]        | 70.4        | 84.9        | 79.8        | 74.3        | 53.9        | 49.8        | 77.5        | 75.9        | 88.5        | 45.6        | 77.1        | 55.3        | 86.9        | 81.7        | 80.9        | 79.6        | 40.1        | 72.6        | 60.9        | 81.2        | 61.5        |
| DEEP_ENS_COCO            | 70.1        | 84.0        | 79.4        | 71.6        | 51.9        | 51.1        | 74.1        | 72.1        | 88.6        | 48.3        | 73.4        | 57.8        | 86.1        | 80.0        | 80.7        | 70.4        | 46.6        | 69.6        | <b>68.8</b> | 75.9        | 71.4        |
| NoC [29]                 | 68.8        | 82.8        | 79.0        | 71.6        | 52.3        | 53.7        | 74.1        | 69.0        | 84.9        | 46.9        | 74.3        | 53.1        | 85.0        | 81.3        | 79.5        | 72.2        | 38.9        | 72.4        | 59.5        | 76.7        | 68.1        |
| Fast R-CNN [14]          | 68.4        | 82.3        | 78.4        | 70.8        | 52.3        | 38.7        | 77.8        | 71.6        | 89.3        | 44.2        | 73.0        | 55.0        | <b>87.5</b> | 80.5        | 80.8        | 72.0        | 35.1        | 68.3        | 65.7        | 80.4        | 64.2        |
| UMICH_FGS_STRUCT         | 66.4        | 82.9        | 76.1        | 64.1        | 44.6        | 49.4        | 70.3        | 71.2        | 84.6        | 42.7        | 68.6        | 55.8        | 82.7        | 77.1        | 79.9        | 68.7        | 41.4        | 69.0        | 60.0        | 72.0        | 66.2        |
| NUS_NIN_C2000 [7]        | 63.8        | 80.2        | 73.8        | 61.9        | 43.7        | 43.0        | 70.3        | 67.6        | 80.7        | 41.9        | 69.7        | 51.7        | 78.2        | 75.2        | 76.9        | 65.1        | 38.6        | 68.3        | 58.0        | 68.7        | 63.3        |
| BabyLearning [7]         | 63.2        | 78.0        | 74.2        | 61.3        | 45.7        | 42.7        | 68.2        | 66.8        | 80.2        | 40.6        | 70.0        | 49.8        | 79.0        | 74.5        | 77.9        | 64.0        | 35.3        | 67.9        | 55.7        | 68.7        | 62.6        |
| NUS_NIN                  | 62.4        | 77.9        | 73.1        | 62.6        | 39.5        | 43.3        | 69.1        | 66.4        | 78.9        | 39.1        | 68.1        | 50.0        | 77.2        | 71.3        | 76.1        | 64.7        | 38.4        | 66.9        | 56.2        | 66.9        | 62.7        |
| R-CNN VGG BB [13]        | 62.4        | 79.6        | 72.7        | 61.9        | 41.2        | 41.9        | 65.9        | 66.4        | 84.6        | 38.5        | 67.2        | 46.7        | 82.0        | 74.8        | 76.0        | 65.2        | 35.6        | 65.4        | 54.2        | 67.4        | 60.3        |
| R-CNN VGG [13]           | 59.2        | 76.8        | 70.9        | 56.6        | 37.5        | 36.9        | 62.9        | 63.6        | 81.1        | 35.7        | 64.3        | 43.9        | 80.4        | 71.6        | 74.0        | 60.0        | 30.8        | 63.4        | 52.0        | 63.5        | 58.7        |
| <b>YOLO</b>              | <b>57.9</b> | <b>77.0</b> | <b>67.2</b> | <b>57.7</b> | <b>38.3</b> | <b>22.7</b> | <b>68.3</b> | <b>55.9</b> | <b>81.4</b> | <b>36.2</b> | <b>60.8</b> | <b>48.5</b> | <b>77.2</b> | <b>72.3</b> | <b>71.3</b> | <b>63.5</b> | <b>28.9</b> | <b>52.2</b> | <b>54.8</b> | <b>73.9</b> | <b>50.8</b> |
| Feature Edit [33]        | 56.3        | 74.6        | 69.1        | 54.4        | 39.1        | 33.1        | 65.2        | 62.7        | 69.7        | 30.8        | 56.0        | 44.6        | 70.0        | 64.4        | 71.1        | 60.2        | 33.3        | 61.3        | 46.4        | 61.7        | 57.8        |
| R-CNN BB [13]            | 53.3        | 71.8        | 65.8        | 52.0        | 34.1        | 32.6        | 59.6        | 60.0        | 69.8        | 27.6        | 52.0        | 41.7        | 69.6        | 61.3        | 68.3        | 57.8        | 29.6        | 57.8        | 40.9        | 59.3        | 54.1        |
| SDS [16]                 | 50.7        | 69.7        | 58.4        | 48.5        | 28.3        | 28.8        | 61.3        | 57.5        | 70.8        | 24.1        | 50.7        | 35.9        | 64.9        | 59.1        | 65.8        | 57.1        | 26.0        | 58.8        | 38.6        | 58.9        | 50.7        |
| R-CNN [13]               | 49.6        | 68.1        | 63.8        | 46.1        | 29.4        | 27.9        | 56.6        | 57.0        | 65.9        | 26.5        | 48.7        | 39.5        | 66.2        | 57.3        | 65.4        | 53.2        | 26.2        | 54.5        | 38.1        | 50.6        | 51.6        |

**Table 3: PASCAL VOC 2012 Leaderboard.** YOLO compared with the full comp4 (outside data allowed) public leaderboard as of November 6th, 2015. Mean average precision and per-class average precision are shown for a variety of detection methods. YOLO is the only real-time detector. Fast R-CNN + YOLO is the forth highest scoring method, with a 2.3% boost over Fast R-CNN.

- VOC 2012 테스트셋 → 57.9% mAP 기록함.
  - 최신 성능보다는 낮지만, 기존 R-CNN(VGG-16 기반) 수준에 근접.
    - 작은 객체(병, 양, TV 등)에 대해 성능 낮음.
    - 고양이, 기차 등 큰 객체에는 성능 우수
- YOLO + Fast R-CNN 조합 → **Fast R-CNN 단독 대비 2.3% mAP 향상**, 공개 리더보드에서 상위 5위권 등극

## 4.5 Generalizability: Person Detection in Artwork

- 일반적인 학술용 데이터셋 → 학습 데이터와 테스트 데이터가 같은 분포.
  - 실제 환경은 그렇지 않은 경우 많음.
- 이를 평가하기 위해, 예술 작품을 대상으로 평가 진행
  - **Picasso Dataset** : 예술 이미지
  - **People-Art Dataset** : 다양한 화풍에서의 사람
  - 각 모델은 자연 이미지(VOC)로 학습하고 예술 이미지에서 테스트



(a) Picasso Dataset precision-recall curves.

|              | VOC 2007<br>AP | Picasso<br>AP | Picasso<br>Best $F_1$ | People-Art<br>AP |
|--------------|----------------|---------------|-----------------------|------------------|
| <b>YOLO</b>  | <b>59.2</b>    | <b>53.3</b>   | <b>0.590</b>          | <b>45</b>        |
| R-CNN        | 54.2           | 10.4          | 0.226                 | 26               |
| DPM          | 43.2           | 37.8          | 0.458                 | 32               |
| Poselets [2] | 36.5           | 17.8          | 0.271                 |                  |
| D&T [4]      | -              | 1.9           | 0.051                 |                  |

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best  $F_1$  score.

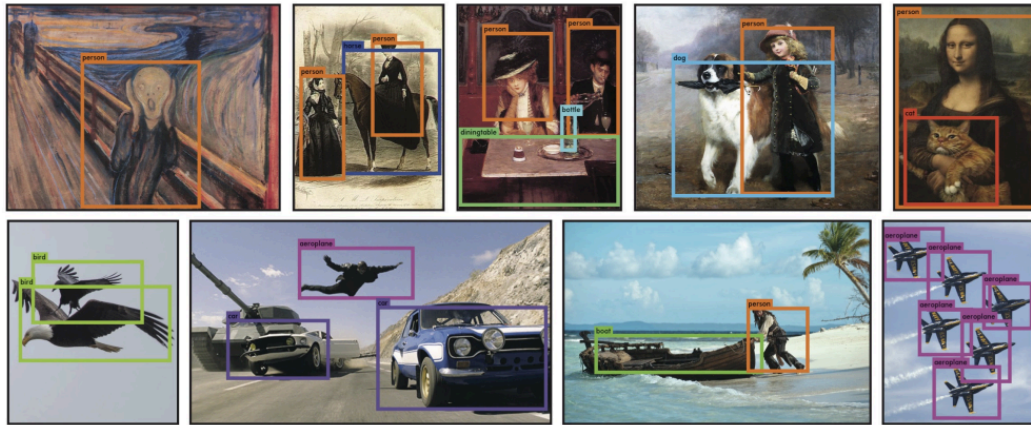
**Figure 5: Generalization results on Picasso and People-Art datasets.**

⇒ YOLO : 도메인 달라져도 **성능 저하 적고, R-CNN보다 훨씬 일반화 잘 됨.**

- DPM : 공간 모델 활용하므로 성능 유지 잘 되는 편.
- YOLO → 객체 크기, 형태, 위치 관계 학습하므로 예술 이미지에서도 강함.

## 5. Real-Time Detection In The Wild

- 빠르고 정확한 객체 탐지기!
  - 다양한 컴퓨터 비전 응용에 적합함.
- 웹캠에 연결하여, 실시간 성능 유지하는지 확인
  - 카메라에서 이미지 가져오고, 탐지 결과를 화면에 출력하는 시간까지 포함하여도 실시간으로 동작함.
  - YOLO는 이미지를 한 장씩 독립적으로 처리하지만, 웹캠과 연결하면 **움직이거나 형태가 바뀌는 객체들을 추적하는 시스템처럼 작동함.**



**Figure 6: Qualitative Results.** YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

## 6. Conclusion

1. **Simple**, 전체 이미지에 대해 직접 학습 가능
2. 탐지 성능과 직접적으로 연관된 손실 함수로 훈련, 모델 전체가 통합적으로(jointly) 학습됨.
3. Fast YOLO → 지금까지 발표된 가장 빠른 범용 객체 탐지기
4. YOLO → 실시간 객체 탐지의 새로운 기준 제시
5. 새로운 도메인에 대해서도 강력한 일반화 성능  
⇒ 빠르고, 강건한 객체 탐지 필요한 다양한 실제 응용에 적합!